Global Hub Analysis & Stochastic Modeling

Ian Eduardo Peralta Hirujo

2026-04-13

Table of contents

1 Busiest Airport Analysis **2**
1.1 Overview . . . 2
1.2 Comparative Traffic Data . . . 2
1.3 Recovery Trajectories . . . 2

2 Monte Carlo Numerical Integration **3**
2.1 Methodology . . . 3
2.2 Convergence Analysis . . . 3

3 Planning and Prompting GenAI Tools **5**
3.1 Tidying and Wrangling Plan . . . 5
3.2 Plan-Informed Prompted Result . . . 6
3.3 Generic Result Comparison . . . 6
3.4 Comparison Narrative . . . 7

4 Reflection **8**

5 Appendix: GenAI Usage **9**

6 Code Appendix **10**
6.1 Airport Analysis Code . . . 10
6.2 Monte Carlo Integration Code . . . 12
6.3 Plan-Informed Calcium Analysis Code . . . 14

Table 1: Annual Passenger Traffic at Global Hubs (2020-2025) in Millions (M)

Airport	IATA Code	2020	2021	2022	2023	2024	2025
Hartsfield–Jackson Atlanta International Airport	ATL	42.9	75.7	93.7	104.7	108.1	106.3
Dubai International Airport	DXB	25.8	29.1	66.1	87.0	92.3	95.2
Dallas Fort Worth International Airport	DFW	39.4	62.5	73.4	81.8	87.8	85.7
Heathrow Airport	LHR	22.1	NA	61.6	79.2	83.9	84.5
Frankfurt Airport	FRA	18.8	24.8	48.9	59.4	61.6	63.2
Beijing Daxing International Airport	PKX	NA	25.1	NA	NA	49.4	53.6

1 Busiest Airport Analysis

1.1 Overview

This analysis examines the annual passenger traffic for six of the world's most prominent airport hubs between 2020 and 2025. By scraping historical data from Wikipedia, we can observe the varying trajectories of recovery following the global travel disruptions of 2020.

1.2 Comparative Traffic Data

As shown in Table 1, the data reveals a sharp divide between domestic centered hubs and international transit points. Domestic hubs like **ATL** and **DFW** maintained high volumes throughout the study period, whereas international hubs like **LHR** and **PKX** suffered from significant data gaps due to regional travel lockdowns. Overall, airports that could rely on local flights experienced a significantly smoother recovery trajectory.

1.3 Recovery Trajectories

The visualization in Figure 1 underscores the "steepness" of recovery for different markets. While **ATL** remains the volume leader, the trajectory for **DXB** represents the most rapid international rebound, nearly quadrupling its 2020 traffic by 2025.

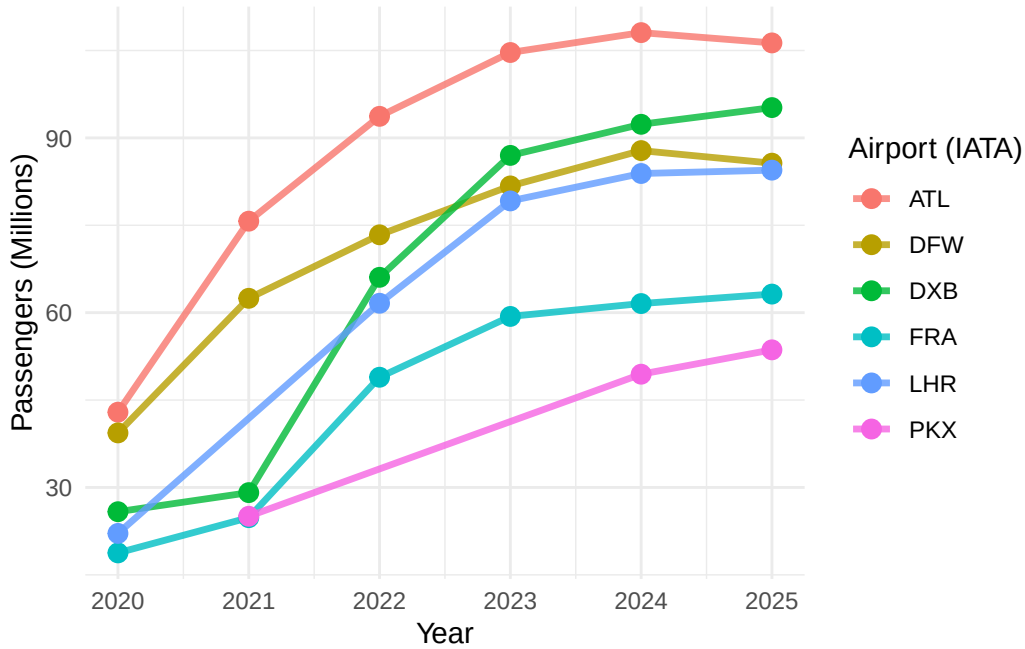


Figure 1: Recovery trajectories of global airport hubs (2020-2025)

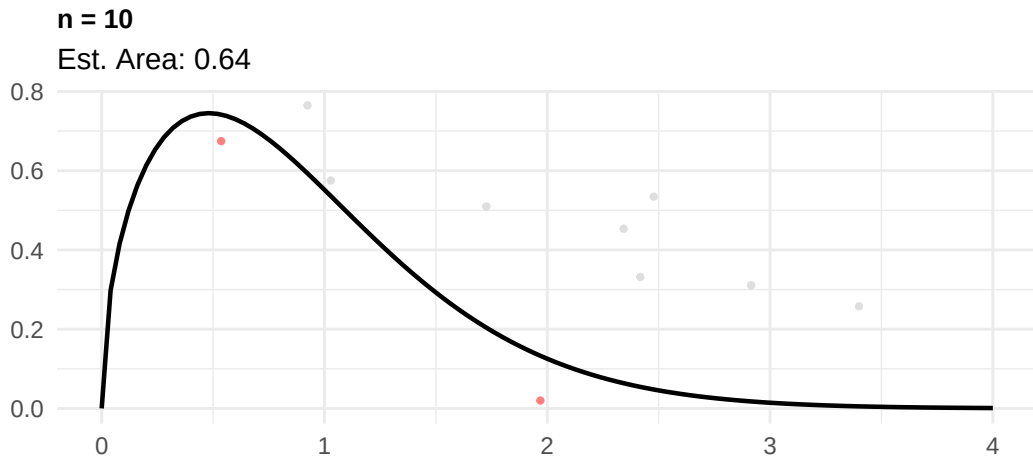
2 Monte Carlo Numerical Integration

2.1 Methodology

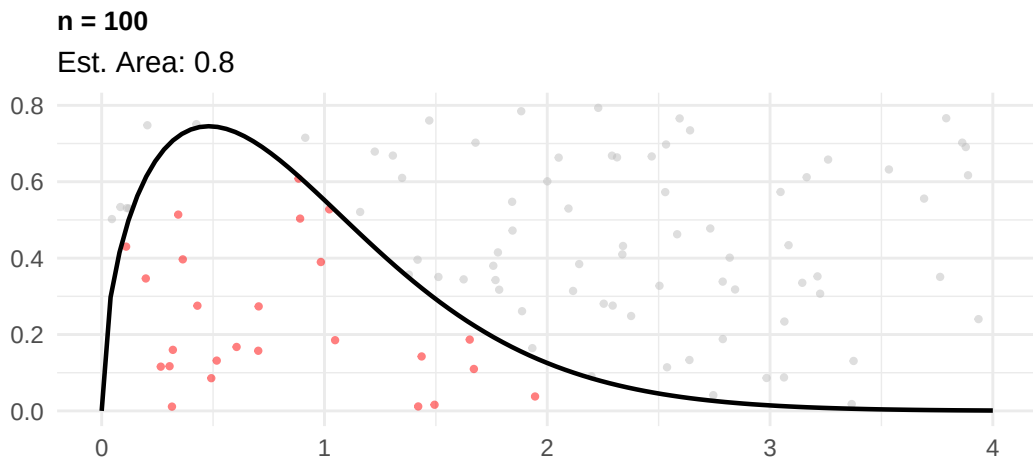
Monte Carlo integration uses random sampling to estimate the area under a curve. For the assigned Weibull PDF (shape = 1.5, scale = 1), we calculate the area by simulating n points in a rectangle of area 3.2 (x in [0, 4], y in [0, 0.8]). The proportion of points landing under the curve provides our estimate.

2.2 Convergence Analysis

As seen in Figures 2-3, the reliability of the estimate is directly tied to the resolution (n). At n = 10, the samples are too sparse to define the curve's shape. However, by n=10,000 (see Figure 3), the "cloud" of points perfectly outlines the distribution. Since this is a probability density function, the exact answer is 1.0. The convergence observed in these sub-figures supports this, as the estimates stabilize near the theoretical value as n increases.

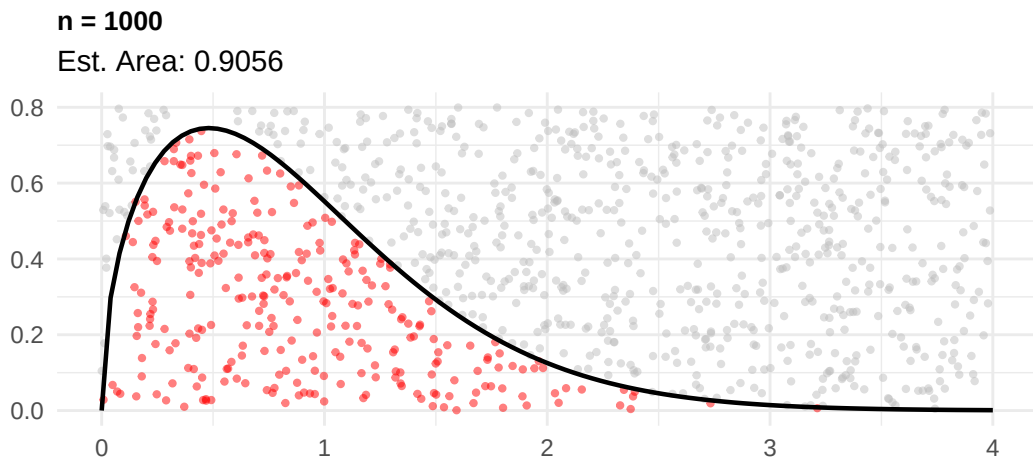


(a) Resolution: n=10

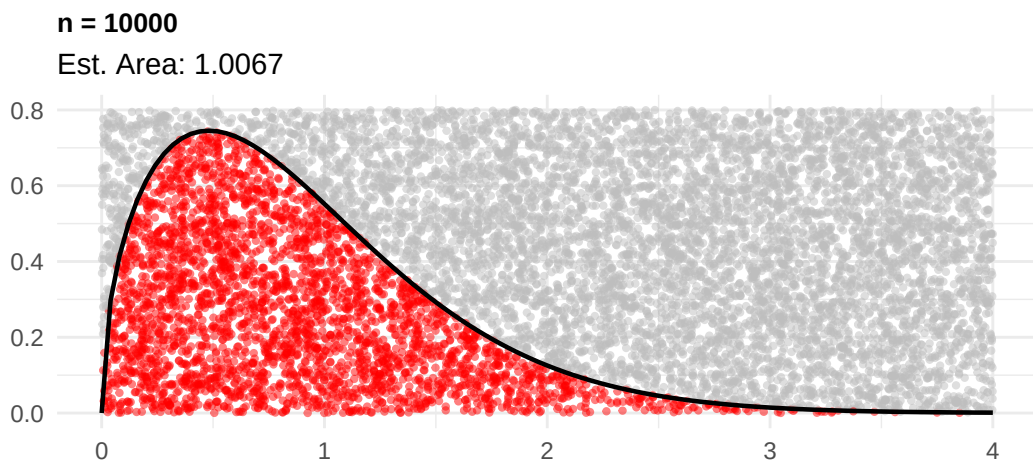


(b) Resolution: n=100

Figure 2



(a) Resolution: n=1000



(b) Resolution: n=10000

Figure 3

3 Planning and Prompting GenAI Tools

3.1 Tidying and Wrangling Plan

Goal: Tidy the calcium.csv dataset and visualize longitudinal ulnar calcium trends across two treatment groups.

Needs: Access to the calcium.csv file, the tidyverse suite for data manipulation, and ggplot2 for longitudinal visualization. Needs: Access to calcium.csv, tidyverse, and ggplot.

Detailed Steps:

1. Load the raw CSV. Note that columns are separated by treatment group but share measurement years.
2. Extract columns 1-4 into a data frame and columns 5-8 into a "Diet/Exercise" data frame.
3. For each data frame, pivot the data from wide to long format so that "Year" becomes a variable and "Calcium" becomes a variable.
4. Combine both data frames and add a categorical "Group" column to distinguish the cohorts.
5. Convert "Year" into an ordered factor to ensure correct temporal plotting order.
6. Generate a longitudinal line plot to observe individual trajectories, overlaid with a mean trend line for each group.

3.2 Plan-Informed Prompted Result

Using the structured plan, Claude successfully identified the treatment groups and produced a professional longitudinal plot. See Code Appendix Section 6.3 for "Plan-Informed Calcium Analysis Code" generated by Claude 4.6 Sonnet.

3.3 Generic Result Comparison

The generic prompt result, provided by the instructor, utilized a simple request: *"Create a data visualization that will help me understand these data."*

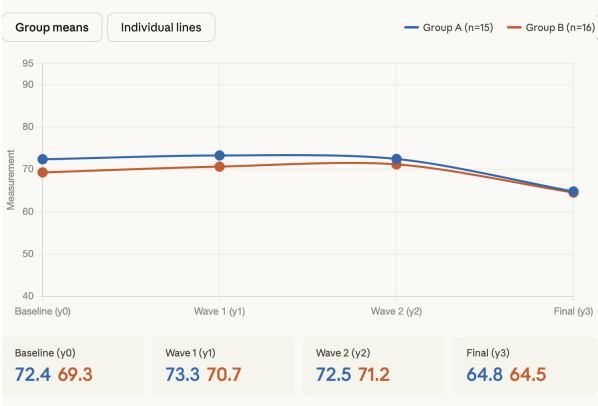


Figure 4: Group Mean Trends

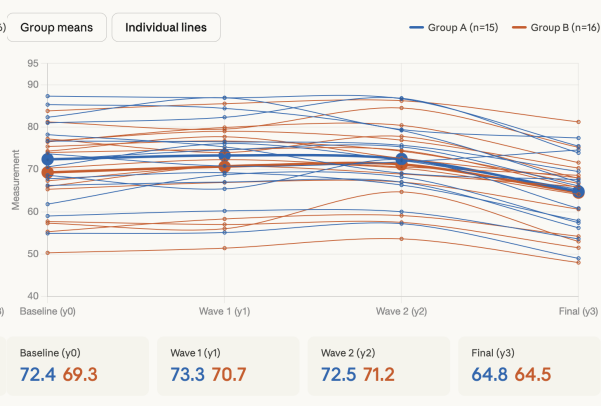


Figure 5: Individual Trajectories

The generic response noted that both groups followed a stable pattern across waves y0 through y2, followed by a sharp drop at y3. It identified that Group A starts higher than Group B but both converge at the final time point.

3.4 Comparison Narrative

The plan-informed prompt produced a much more contextually accurate representation than the generic prompt. While the generic result in Figure 4 correctly identified "Group A" and "Group B," it relied on Claude's ability to "guess" that the columns were grouped by cohort. By explicitly instructing the tool to separate the first four columns from the last four in my plan, I ensured that Figure 6 strictly respected the study design.

Furthermore, my plan-informed result utilized the specific labels "Diet/Exercise" and "Null," making the visualization immediately actionable for a researcher. This exercise reveals that while GenAI is capable of parsing messy data, a well-structured "Steps" logic prevents errors in cohort assignment and results in much higher quality, professional outputs.

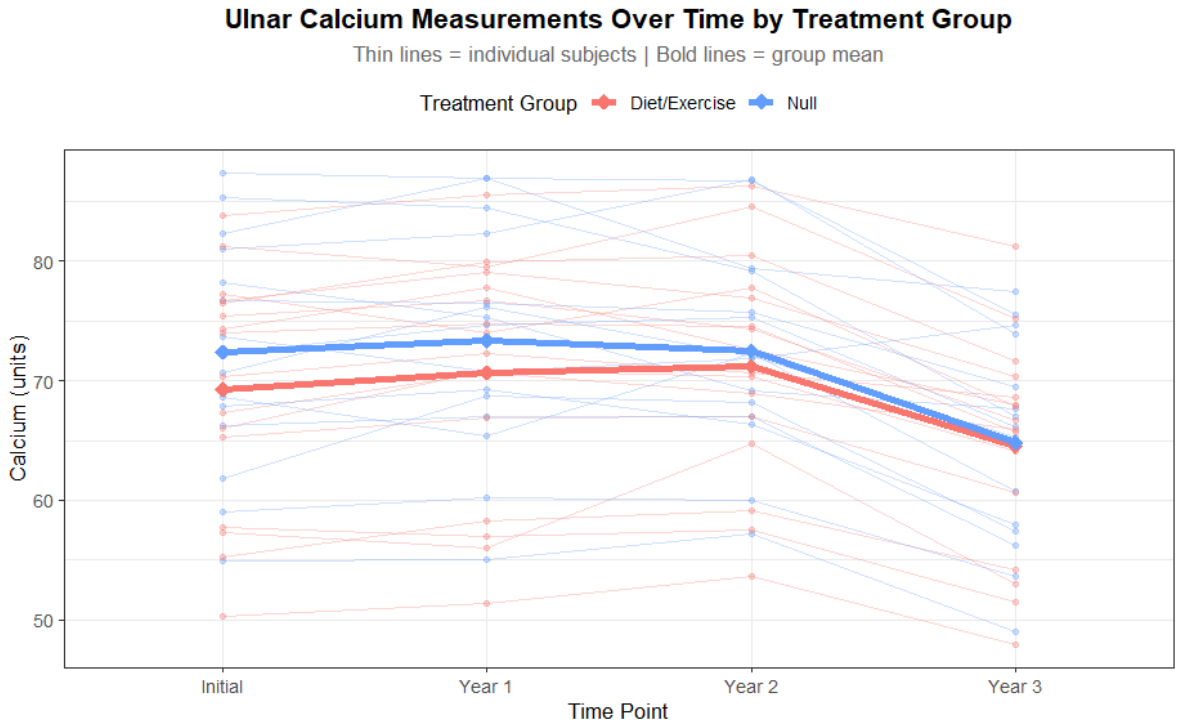


Figure 6: Plan-Informed Plot

4 Reflection

As a Data Science student, building this reproducible document has solidified my understanding of the data science lifecycle. Moving away from patchwork to Quarto's native layout-ncol in Figures 2-3 taught me how to generate quality figures without external dependencies. The process of scraping and tidying data taught me the importance of transparent, reproducible pipelines in a field where raw data is often unstructured. Furthermore, the contrast between generic and planned prompting highlighted that while AI can wrangle data, it still relies on a researcher's explicit logic to avoid the obstacles of algorithmic guesswork . Ultimately, mastering these reproducible techniques has improved my perspective from simply executing code to delivering more professional and insightful analysis that satisfies to academic standards.

5 Appendix: GenAI Usage

- **Tool:** Claude 4.6 Sonnet
- **Date:** April 13, 2026
- **Prompt (Calcium Plan):**

  I am working on a longitudinal study analysis involving ulnar calcium measurements. I have a file called calcium.csv. My Goal: Tidy this dataset and visualize calcium trends over four years across two treatment groups (Null vs. Diet/Exercise). Data Context: The first 4 columns are for the Null group and the next 4 are for the Diet/Exercise group.

  My Plan: Load the CSV. Separate columns 1-4 into a data frame and columns 5-8 into a Diet/Exercise data frame. Pivot both from wide to long format so I have Year and Calcium columns. Combine them back together with a new column identifying the Treatment Group. Create a longitudinal ggplot visualization to show the difference between groups. Please provide the R code to execute this plan.

- **Response:** (See Code Appendix Section 6.3)
  - **Subject IDs** are assigned by row index within each group. This is important for `geom_line()` to correctly connect each person's measurements across time without crossing over between subjects.
  - **The `interaction()` call** in `aes(group = ...)` ensures ggplot treats each subject *within* each treatment as a separate trajectory, preventing lines from connecting across groups.
  - **`stat_summary()`** is layered on top to draw bold group-mean lines, turning this into a spaghetti plot with a clear summary overlay — a standard approach for longitudinal data visualization.
  - If your CSV columns have actual header names, you can replace the `colnames()` assignments with `select()` calls using those names instead.

6 Code Appendix

6.1 Airport Analysis Code

```
target_iata <- c("ATL", "FRA", "PKX", "DFW", "DXB", "LHR")
wiki_url <-
  "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"

wp_page <- read_html(wiki_url)
all_tables <- html_elements(wp_page, "table.wikitable")

process_airport_table <- function(table_index, year_label) {
  all_tables[[table_index]] |>
    html_table() |>
    rename(Airport = matches("Airport"),
           Code = matches("Code|IATA|ICAO"),
           Passengers = matches("Passenger|Total")) |>
    select(Airport, Code, Passengers) |>
    mutate(
      Year = year_label,
      IATA = str_sub(Code, 1, 3),
      Passengers = parse_number(as.character(Passengers))
    ) |>
    filter(IATA %in% target_iata)
}

busiest_airports_tidy <- bind_rows(
  process_airport_table(1, 2025), process_airport_table(2, 2024),
  process_airport_table(3, 2023), process_airport_table(4, 2022),
  process_airport_table(5, 2021), process_airport_table(6, 2020)
) |>
  select(Airport, IATA, Year, Passengers) |>
  arrange(IATA, Year)

table_data <- busiest_airports_tidy |>
  mutate(Passengers_M = Passengers / 1e6) |>
  select(Airport, IATA, Year, Passengers_M) |>
  pivot_wider(names_from = Year, values_from = Passengers_M) |>
  arrange(desc(`2025`))

table_data |>
```

```r
  gt() |>
  fmt_number(columns = matches("202"), decimals = 1) |>
  cols_label(IATA = "IATA Code") |>
  tab_options(table.width = pct(100), table.font.size = "9pt") |>
  cols_width(Airport ~ pct(35))

plot_data <- busiest_airports_tidy |>
  mutate(Passengers_M = Passengers / 1e6, Year = as.numeric(Year))

ggplot(plot_data, aes(
  x = Year, y = Passengers_M, color = IATA, group = IATA)) +
  geom_line(linewidth = 1.2, alpha = 0.8) +
  geom_point(size = 3) +
  theme_minimal() +
  scale_x_continuous(breaks = 2020:2025) +
  labs(x = "Year", y = "Passengers (Millions)", color = "Airport (IATA)")
```

6.2 Monte Carlo Integration Code

```
# Generating (x, y) pairs within specified bounds
generate_mc_simulation <- function(n, x_min, x_max, y_min, y_max) {

  # Randomly sample n coordinates using a uniform distribution
  x_coords <- runif(n, min = x_min, max = x_max)
  y_coords <- runif(n, min = y_min, max = y_max)

  # Join these two samples together into one data frame
  sim_data <- data.frame(
    x = x_coords,
    y = y_coords,
    n_resolution = n
  )

  return(sim_data)
}

make_mc_plot <- function(n_val) {

  # Generate simulation data
  sim_data <- generate_mc_simulation(
    n = n_val, x_min = 0, x_max = 4, y_min = 0, y_max = 0.8)

  # Wrangle and Calculate
  wrangled_data <- sim_data |>
    mutate(status = if_else(y <= dweibull(
      x, shape = 1.5, scale = 1), "Below/On", "Above"))

  estimate <- 3.2 * mean(wrangled_data$status == "Below/On")

  # Create the plot object
  p <- ggplot(wrangled_data, aes(x = x, y = y)) +
    geom_point(aes(color = status), alpha = 0.5, size = 0.8) +
    stat_function(fun = dweibull, args = list(shape = 1.5, scale = 1),
                  xlim = c(0, 4), color = "black", linewidth = 0.8) +
    scale_color_manual(values = c("Below/On" = "red", "Above" = "gray")) +
    theme_minimal() +
    labs(title = paste("n =", n_val),
         subtitle = paste("Est. Area:", round(estimate, 4)),
         x = NULL, y = NULL) +
```

```r
    theme(legend.position = "none", plot.title = element_text(
      size = 10, face = "bold"))

  return(p)
}
make_mc_plot(10)
make_mc_plot(100)
make_mc_plot(1000)
make_mc_plot(10000)
```

6.3 Plan-Informed Calcium Analysis Code

```
library(tidyverse)

# 1. Load the CSV
calcium_raw <- read.csv("calcium.csv")

# 2. Separate into Null and Diet/Exercise data frames
null_df <- calcium_raw[, 1:4]
diet_df  <- calcium_raw[, 5:8]

# 3. Standardize column names and add subject IDs, then pivot to long format
colnames(null_df) <- c("Initial", "Yr1", "Yr2", "Yr3")
colnames(diet_df) <- c("Initial", "Yr1", "Yr2", "Yr3")

null_df$Subject <- 1:nrow(null_df)
diet_df$Subject <- 1:nrow(diet_df)

null_long <- null_df %>%
  pivot_longer(
    cols      = c("Initial", "Yr1", "Yr2", "Yr3"),
    names_to  = "Year",
    values_to = "Calcium"
  )

diet_long <- diet_df %>%
  pivot_longer(
    cols      = c("Initial", "Yr1", "Yr2", "Yr3"),
    names_to  = "Year",
    values_to = "Calcium"
  )

# 4. Add treatment group labels and combine
null_long$Treatment <- "Null"
diet_long$Treatment <- "Diet/Exercise"
combined_df <- bind_rows(null_long, diet_long)

# Re-order Year factor so the x-axis displays chronologically
combined_df$Year <- factor(
  combined_df$Year,
  levels = c("Initial", "Yr1", "Yr2", "Yr3"),
  labels = c("Initial", "Year 1", "Year 2", "Year 3")
```

```r
)

# 5. Spaghetti plot with group mean overlay
ggplot(combined_df, aes(x = Year, y = Calcium, group = interaction(
  Subject, Treatment), color = Treatment)) +

  # Individual subject trajectories (thin, semi-transparent)
  geom_line(alpha = 0.3, linewidth = 0.5) +
  geom_point(alpha = 0.3, size = 1.2) +

  # Group mean trajectories (bold overlay)
  stat_summary(
    aes(group = Treatment),
    fun      = mean,
    geom     = "line",
    linewidth = 1.8,
    linetype  = "solid"
  ) +
  stat_summary(
    aes(group = Treatment),
    fun   = mean,
    geom  = "point",
    size  = 4,
    shape = 18
  ) +

  # Styling
  scale_color_manual(values = c(
    "Null" = "#619CFF", "Diet/Exercise" = "#F8766D")) +
  labs(
    title    = "Ulnar Calcium Measurements Over Time by Treatment Group",
    subtitle = "Thin lines = individual subjects | Bold lines = group mean",
    x        = "Time Point",
    y        = "Calcium (units)",
    color    = "Treatment Group"
  ) +
  theme_bw(base_size = 13) +
  theme(
    plot.title    = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5, color = "grey40"),
    legend.position = "top"
    )
```